

**МИНОБРНАУКИ РОССИИ**  
**САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ**  
**ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ**  
**«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)**  
**Кафедра МО ЭВМ**

**ИНДИВИДУАЛЬНОЕ ДОМАШНЕЕ ЗАДАНИЕ**  
**по дисциплине «Введение в нереляционные базы данных»**  
**Тема: Каталог схем для вязания и вышивки с аналитикой**

Студент гр. 2300

Сысуев М.Ю.

Преподаватель

Заславский М.М.

Санкт-Петербург

2025

## ЗАДАНИЕ

Студентка Сысуев М.Ю.

Тема проекта: Каталог схем для вязания и вышивки с аналитикой.

Исходные данные:

Задача - реализовать сервис, где можно создавать, хранить и редактировать схемы для вязания / вышивки, а также проводить их анализ на предмет объема необходимого материала, сложности, параметров изделия.

Содержание пояснительной записки:

«Содержание», «Введение», «Сценарии использования», «Модель данных», «Разработанное приложение», «Выводы», «Приложения», «Список использованных источников».

Предполагаемый объем пояснительной записки:

Не менее 20 страниц.

Дата выдачи задания: 03.02.2025

Дата сдачи реферата: 05.06.2025

Дата защиты реферата: 05.06.2025

Студент гр. 2300

Сысуев М.Ю.

Преподаватель

Заславский М.М.

## АННОТАЦИЯ

В рамках ИДЗ было разрабатывается веб-приложение для публикации и просмотра схем для вышивания. Приложение включает функционал для просмотра и создания схем, а также добавления новых способов вышивания(крючки стежки). Реализована система фильтрации и поиска для всех существующих в системе сущностей.

При разработке приложения были использованы следующие языки программирования: Python и HTML а также НСУБД Mongo и Docker.

Исходный код приложения доступен по ссылке:  
[https://github.com/moevm/nosql1h25-needlework/tree/Sysuev\\_Mikhail](https://github.com/moevm/nosql1h25-needlework/tree/Sysuev_Mikhail).

## СОДЕРЖАНИЕ

|      |                                                          |    |
|------|----------------------------------------------------------|----|
|      | Введение                                                 | 5  |
| 1    | Макет UI                                                 | 7  |
| 2    | Сценарий использования                                   | 7  |
| 2.1. | Работа с пользователем-действующим лицом                 | 7  |
| 3.   | Нереляционная модель                                     | 14 |
| 3.1. | Описание назначений коллекций, типов данных и сущностей. | 14 |
| 3.2. | Объём памяти для хранения всех видов объектов            | 17 |
| 3.3. | Избыточность данных                                      | 17 |
| 3.4. | Примеры запросов                                         | 18 |
| 4.   | Реляционная модель                                       | 21 |
| 4.1. | Описание назначений коллекций, типов данных и сущностей  | 21 |
| 4.2. | Избыточность данных                                      | 24 |
| 4.3  | Примеры запросов                                         | 25 |
| 5    | Сравнение моделей                                        | 28 |
| 6    | Разработанное приложение                                 | 30 |
| 6.1. | Краткое описание                                         | 30 |
| 6.2. | Использованные технологии                                | 30 |
| 6.3. | Снимки экрана приложения                                 | 30 |
|      | Выводы                                                   | 35 |
|      | Документация по сборке и развертыванию приложения        | 37 |
|      | Инструкция для пользователя                              | 37 |
|      | Литература                                               | 38 |

## **ВВЕДЕНИЕ**

### **Актуальность проблемы**

В современном мире рукоделие, особенно вышивание, остается популярным хобби, которое помогает расслабиться, выразить творческие идеи и создавать уникальные изделия. Однако поиск качественных схем для вышивания может быть сложным и отнимать много времени.

### **Постановка задачи**

Создание веб-сервиса «needlework», который будет решать следующие задачи:

1. Просмотр схем:
  - По умолчанию схемы расположены в порядке от новой к старой.
  - Можно искать схемы используя многочисленные фильтры и сортировки а так же поиск.
2. Просмотр стежков:
  - Поиск всех стежков
  - Различные фильтры и поиск по названию.
3. Функции для администрирования системы:
  - Массовый экспорт и импорт
  - Кнопка отчистки базы данных

### **Предлагаемое решение**

Для реализации backend и frontend частей веб-приложения используется Python с фреймворком FastAPI и jinja Для хранения данных применяется НСУБД Mongo, а для контейнеризации и развертывания – Docker.

### **Качественные требования к решению**

Решение должно быть удобным, производительным, надежным и легко

расширяемым, с высокой степенью взаимодействия между пользователями, удобным механизмом поиска и фильтрации, а также возможностью быстрого развертывания на различных платформах благодаря использованию Docker.

## МАКЕТ UI

На рис. 1 представлен разработанный макет UI веб-приложения.

Ссылка на макет в Figma

<https://www.figma.com/design/5b2pXrdkpsEf1mgQG9nRxU/nosql-frontend?node-id=0-1&p=f&t=mnle89OZyYr7prFz-0>.

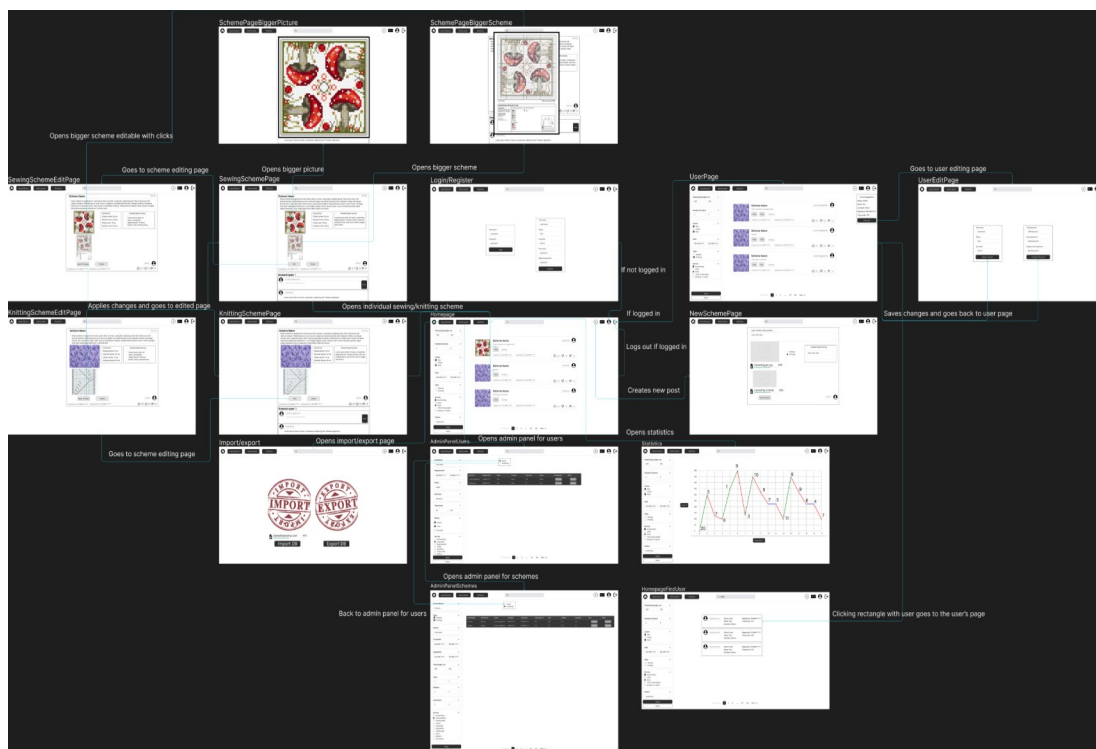


Рис. 1 – Макет UI

## СЦЕНАРИИ ИСПОЛЬЗОВАНИЯ

### Работа с пользователем-действующим лицом

#### Главная страница

Действующее лицо: пользователь

Основной сценарий:

Пользователь заходит на сайт и попадает на главную страницу.

На главной странице отображаются схемы для вышивания или вязания от всех пользователей. По умолчанию схемы сортируются по дате добавления (новые сверху).

Слева находится панель с фильтрами для сортировки и поиска схем:

Количество цветов (от ... до ...).

Выбор цветов (список).

Дата публикации (от ... до ...).

Тип работы (вязание, вышивка).

Пользователь сделавший схему.

Количество лайков (от ... до ...).

Сортировка:

По лайкам.

По дате публикации.

По длине используемой нити.

По количеству цветов.

Порядок сортировки:

По умолчанию (по возрастанию).

Обратный (по убыванию).

Автор (поле для ввода username автора схем)

Пользователь может применять фильтры и сортировку для удобного просмотра схем.

Альтернативный сценарий:

Если пользователь не авторизован, он может просматривать схемы, но не может лайкать, комментировать или загружать свои схемы.

## **Панель навигации**

Действующее лицо: пользователь

Основной сценарий:

В верхней части всех страниц сайта находится навигационная панель.

На панели отображаются следующие элементы:

Кнопка домашней страницы: Переход на главную страницу.

Кнопка "Создать новый пост": Видна только авторизованным



пользователям. При нажатии открывается страница для загрузки новой схемы.

Кнопка "Выйти": Видна только авторизованным пользователям. При нажатии пользователь выходит из аккаунта.

Импорт/Экспорт: Направляет на страницу импорта/экспорта данных.

Альтернативные сценарии:

Для неавторизованных пользователей:

Кнопка "Создать новый пост" не отображается.

Кнопка "Выйти" отсутствует.

Кнопки импорта, экспорта не отображаются.

## **Авторизация**

Действующее лицо: пользователь

Основной сценарий:

Пользователь нажимает на иконку пользователя.

Открывается окно авторизации и регистрации.

В левой части окна пользователь может ввести:

Логин.

Пароль.

Пользователь нажимает кнопку "Login".

Если данные верны, пользователь авторизуется и получает доступ к функционалу (лайки, комментарии, загрузка схем).

Альтернативный сценарий:

Если данные введены некорректно, пользователь видит сообщение об ошибке (например, "Неверный логин или пароль").

## **Регистрация**

Действующее лицо: пользователь

Основной сценарий:

Пользователь нажимает на иконку пользователя.

Открывается окно входа и регистрации.

В правой части окна пользователь вводит:

Логин.

Имя.

Фамилию.

Пароль.

Пользователь нажимает кнопку "Register".

Если данные введены корректно, создаётся новый аккаунт, и пользователь автоматически авторизуется.

Альтернативный сценарий:

Если данные введены некорректно, пользователь видит сообщение об ошибке.

Если логин уже занят, пользователь видит сообщение об ошибке (например, "Такой пользователь уже существует").

### **Поиск пользователя**

Действующее лицо: пользователь(авторизованный/неавторизованный)

Основной сценарий:

Пользователь переходит на страницу поиска пользователей и вводит имя, фамилию или юзернейм.

И еще он может ввести различные фильтры или отсортировать пользователей.

Сайт ищет в каталоге нужного пользователя и выдает его профиль.

### **Открытие определенной схемы**

Действующее лицо: пользователь

Основной сценарий:

Пользователь, найдя нужную ему схему, нажимает кнопку View.

Сайт открывает окно просмотра этой схемы.

### **Оценивание поста**

Действующее лицо: авторизованный пользователь

Основной сценарий:

Если пользователь авторизован, он может поставить посту лайк/дизлайк, нажав на определенную кнопку.

Альтернативный сценарий:

Если пользователь не авторизован, то при попытке оценки выдаст ошибку "Недоступно неавторизованным пользователям".

### **Комментирование**

Действующее лицо: авторизованный пользователь

Основной сценарий:

Если пользователь авторизован, то он может написать комментарий к схеме и опубликовать его, нажав на кнопку Post.

Альтернативный сценарий:

Если пользователь не авторизован, то он не может опубликовать свой комментарий, у него нет кнопки Post.

### **Страница загрузки новой схемы**

Действующее лицо: авторизованный пользователь

Основной сценарий:

Пользователь авторизован и нажимает на кнопку "Создать новый пост" в верхней навигационной панели.

Пользователь перенаправляется на страницу загрузки новой схемы.

На странице отображается форма для загрузки схемы:

Поле "Название схемы": Пользователь вводит название схемы.

Поле "Описание схемы": Пользователь пишет описание схемы (например,

материалы, сложность, инструкции).

Выбор типа работы: Пользователь выбирает тип работы из списка:

Вязание.

Вышивание.

Поле "Комментарий автора": Пользователь может добавить дополнительный комментарий.

Поле для загрузки изображения: Пользователь загружает изображение схемы (например, фотография готового изделия или пример).

Поле для загрузки файла схемы: Пользователь загружает файл схемы.

После заполнения всех полей пользователь нажимает кнопку "Post Scheme".

Если все данные введены корректно, схема публикуется и появляется в списке на главной странице и на странице пользователя.

Альтернативный сценарий:

Если пользователь не заполнил обязательные поля (например, название схемы или файл схемы), появляется сообщение об ошибке (например, "Заполните все обязательные поля").

Если загруженный файл не соответствует требованиям (например, слишком большой размер или недопустимый формат), появляется сообщение об ошибке (например, "Недопустимый формат файла").

### **Импорт/Экспорт**

Действующее лицо: авторизованный пользователь (администратор)

Основной сценарий:

Пользователь на панели навигации нажимает на кнопку Import/Export.

Сайт перенаправляет его на страницу импорта и экспорта.

Пользователь может нажать на кнопку Import DB и импортировать данные.

Пользователь может нажать на кнопку Export DB и экспортировать

данные.

Так же для тестирования кнопка очистить бд.

Альтернативный сценарий:

Если пользователь не авторизован или авторизован, но не является админом, он не видит кнопки Import/Export и не может выполнить импорт или экспорт данных с сайта.

Вывод: операция чтения преобладают над операциями записи в системе.

# НЕРЕЛЯЦИОННАЯ МОДЕЛЬ

## Графическое представление модели

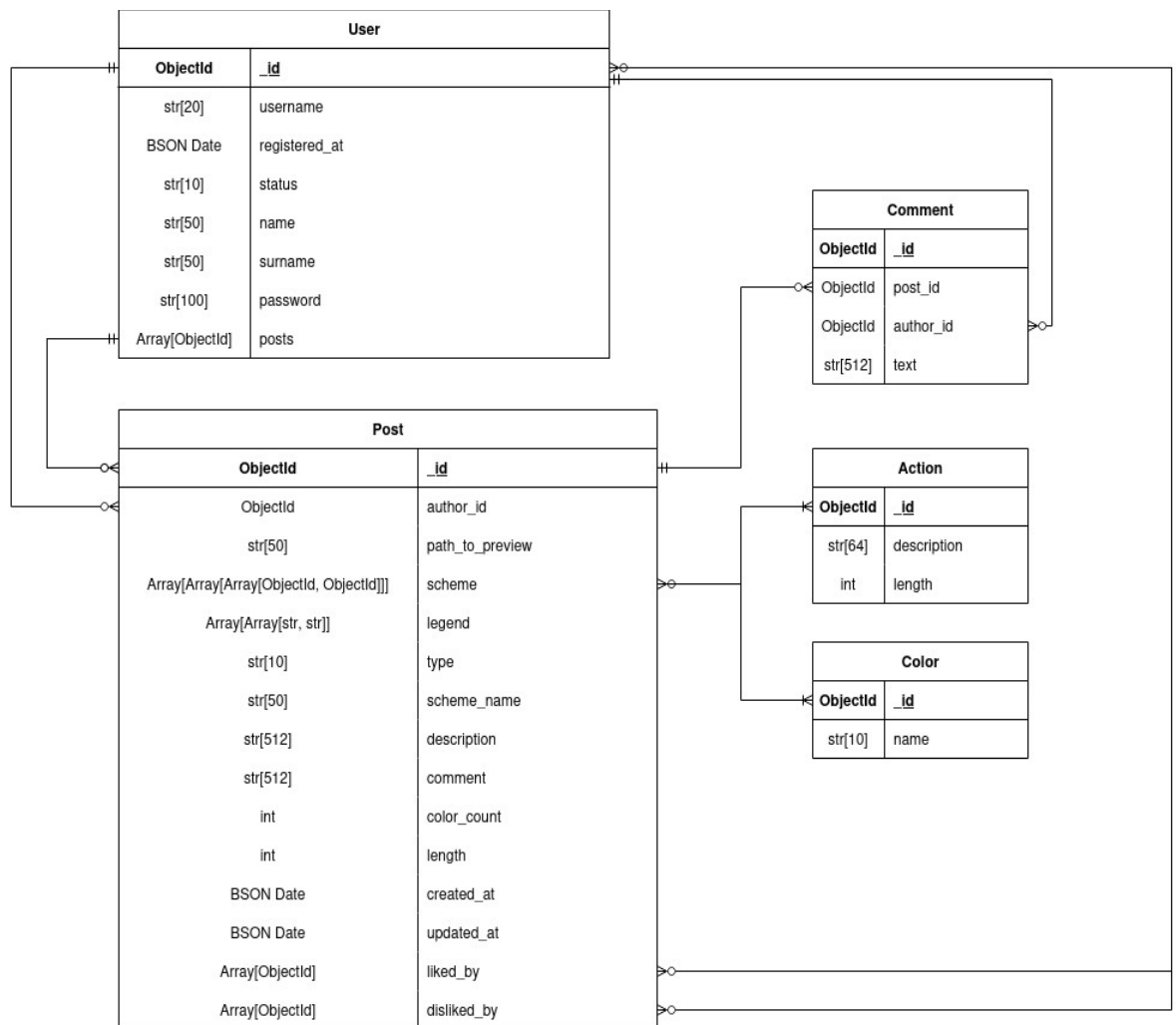


Рис. 2 – Графическое представление нереляционной модели

## Описание назначений коллекций, типов данных и сущностей

### Типы данных

Сущности используют следующие типы данных.

#### Базовые

- **string**: текст.
- **int**: целое число.
- **datetime**: дата и время.

- **Array:** Массив.
- **ObjectId:** id сущности.
- **BINDATA:** для хранения картинок.

## Сущности и коллекции

### Коллекция posts

Поля и размеры:

- `_id`: 12 байт (ObjectId)
- `preview_image_id`: 2097152 байт (BINDATA)
- `scheme`: 80000 байт `Array[Array[Tuple[int, int]]]`
- `legend`: 10000 байт (`Array(Array(str, str))`)
- `type`: 10 байт (строка)
- `"scheme_name"`: 200 байт (строка),
- `description`: 2000 байт (строка)
- `comment`: 512 байт (строка)
- `author_id`: 12 байт (ObjectId)
- `color_count`: 4 (число)
- `length`: 4 (число)
- `created_at`: 8 байт (BSON)
- `updated_at`: 8 байт (BSON)
- `likes`: 12000 байт (массив)
- `dislikes`: 12000 байт (массив)

### Коллекция users

Поля и размеры:

- `_id`: 12 байт (ObjectId)
- `username`: 20 байт (строка)
- `registration_at`: 8 байт (BSON)

- status: 10 байт (строка)
- name: 50 байт (строка)
- surname: 50 байт (строка)
- password: 100 байт (строка)
- posts: 120 байт (Array(ObjectId))

### **Коллекция comments**

Поля и размеры:

- \_id: 12 байт (ObjectId)
- post\_id: 12 байт (ObjectId)
- author\_id: 12 байт (ObjectId)
- text: 512 байт (строка)

### **Коллекция actions**

Поля и размеры:

- \_id: 12 байт (ObjectId)
- description: 64 байт (строка)
- length: 4 (число)

### **Comment**

Данные о комментарии.

Поля (208 байт):

- **text:** string (~200 байт) - текст комментария
- **date:** datetime (8 байт) - дата и время отправки комментария

### **Коллекция colors**

Поля и размеры:

- \_id: 12 байт (ObjectId)
- name: 10 байт (строка)



## **Объём памяти для хранения всех видов объектов**

### **Средний размер данных**

#### **Коллекция posts**

12 (\_id) + 2097152 (preview) + 80000 (scheme) + 10000 (legend) + 200 (scheme\_name) + 10 (type) + 2000 (description) + 512 (comment) + 12 (author\_id) + 4 (color\_count) + 4 (length) + 8 (created\_at) + 8 (updated\_at) + 12000 (likes) + 12000 (dislikes) = 2,213,922 байт

#### **Коллекция users**

12 (\_id) + 20 (username) + 8 (registration\_at) + 10 (status) + 50 (name) + 50 (surname) + 100 (password) + 120 (posts) = 370 байт

#### **Коллекция comments**

12 (\_id) + 12 (post\_id) + 12 (author\_id) + 512 (text) = 548 байт

#### **Коллекция actions**

12 (\_id) + 64 (description) + 4 (length) = 80 байт

#### **Коллекция colors**

12 (\_id) + 10 (name) = 22 байт

### **Избыточность данных**

- Избыточные данные

Длина 4 байт

Комментарий 512 байт

Дата создания 8 байт

Дата обновления 8 байт

- Чистый объем

$(2.213.922 - 4 - 512 - 8 - 8) * N = 2.213.390 * N$

- Коэффициент избыточности

$2.214.942 / 2.213.390 = 1,0007$

## Направление роста модели

Направление роста

| Операция                 | Прирост данных |
|--------------------------|----------------|
| Регистрация пользователя | +117,4 КБ      |
| Добавление новой схемы   | +2,1 МБ        |
| + 1000 комментариев      | +535 МБ        |

Пример роста для 1 млн постов

| Компонент                           | Объём   |
|-------------------------------------|---------|
| Посты                               | 2,01 ТБ |
| Пользователь                        | 112 ГБ  |
| Комментарии (+1000 к каждому посту) | 510 ТБ  |

## Примеры запросов

**Запросы к коллекции posts**

**Создание поста**

```
await db.posts.insert_one({
  "preview_image_id": await upload_image_from_file(
    fs,
    images_dir / "photo1.jpg",
    "цветочный_узор.jpg"
  ),
  "scheme": [[(1, 10), (2, 15)], [(3, 20), (4, 25)]],
  "legend": [("1", "красный"), ("2", "синий")],
```

```
"type": "вышивка",
"scheme_name": "Цветочный узор",
"description": "Схема для вышивки цветов",
"comment": "Используйте нитки DMC",
"author_id": user_ids[0],
"created_at": datetime.now(timezone.utc),
"updated_at": datetime.now(timezone.utc),
"likes": [],
"dislikes": []
})
```

#### **Добавление лайка**

```
await db.posts.update_one(
    {"_id": post_id},
    {"$addToSet": {"likes": user_id}, "$pull": {"dislikes": user_id}}
)
```

#### **Добавление дизлайка**

```
await db.posts.update_one(
    {"_id": post_id},
    {"$addToSet": {"dislikes": user_id}, "$pull": {"likes": user_id}}
)
```

#### **Получение информации о посте**

```
await db.posts.find_one({"_id": post_id})
```

#### **Получение всех постов**

```
await db.posts.find().to_list(None)
```

### **Создание индекса**

```
await db.posts.create_index("author_id")
```

### **Запросы к коллекции comments**

#### **Создание комментария**

```
await db.comments.insert_one({  
    "post_id": post_id,  
    "author_id": author_id,  
    "text": text[:512]  
})
```

#### **Получение комментариев для поста**

```
await db.comments.find({"post_id": post_id}).to_list(None)
```

### **Создание индекса**

```
await db.comments.create_index("post_id")
```

### **Запросы к коллекции users**

#### **Создание пользователя**

```
await db.users.insert_one({  
    "username": username[:20],  
    "registration_at": datetime.now(timezone.utc),  
    "status": status[:10],  
    "name": name[:50],  
    "surname": surname[:50],  
    "password": password[:100],  
    "posts": []  
})
```

### **Получение информации о пользователе**

```
await db.users.find_one({"_id": user_id})
```

### **Получение всех пользователей**

```
await db.users.find().to_list(None)
```

### **Обновление списка постов пользователя**

```
await db.users.update_one(  
    {"_id": author_id},  
    {"$push": {"posts": result.inserted_id}}  
)
```

### **Создание индекса**

```
await db.users.create_index("username", unique=True)
```

### **Запросы к коллекции colors**

#### **Создание цвета**

```
await db.colors.insert_one(  
    "name": name[:10]  
)
```

#### **Получение всех цветов**

```
await db.colments.find().to_list(None)
```

#### **Создание индекса**

```
await db.colors.create_index("name")
```

# РЕЛЯЦИОННАЯ МОДЕЛЬ

## Графическое представление модели

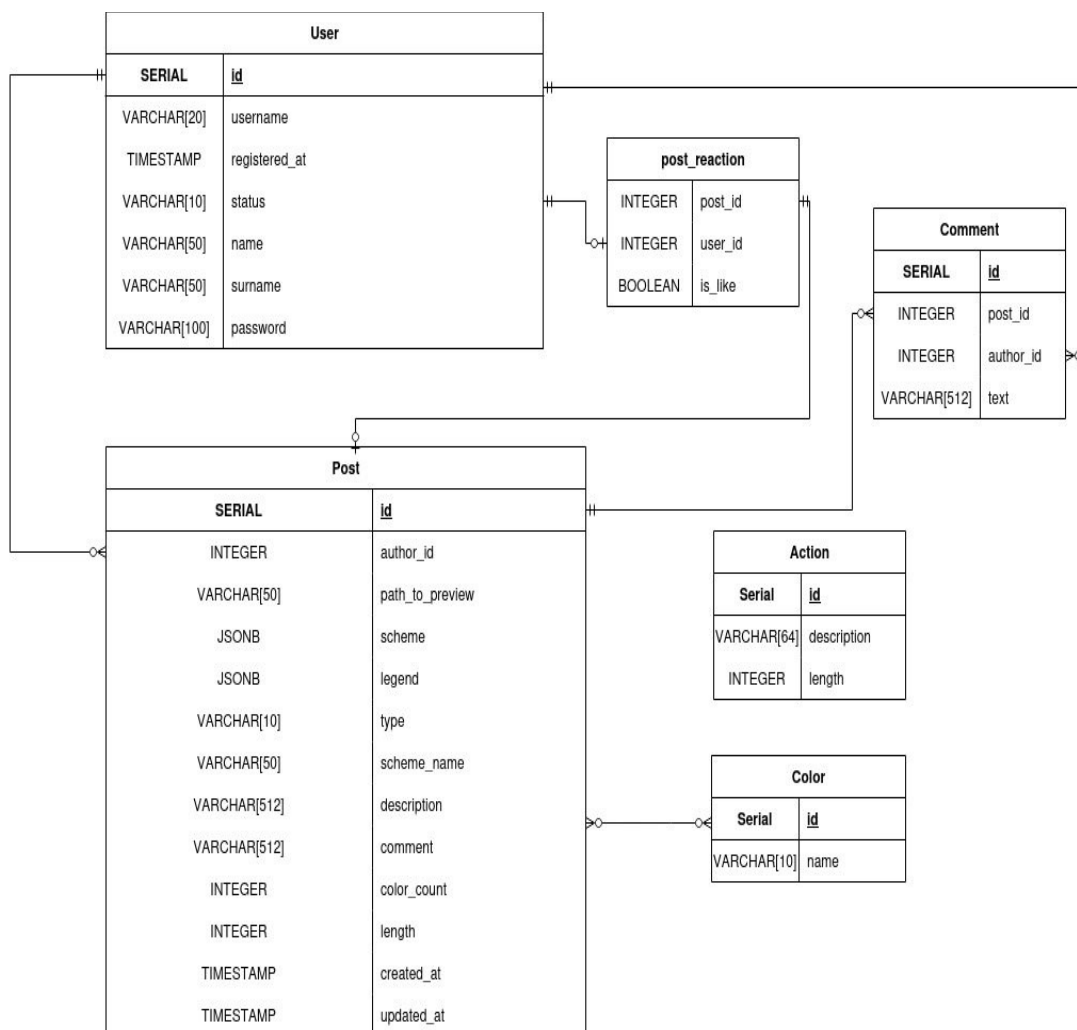


Рис. 3 – Графическое представление реляционной модели данных  
Описание назначений коллекций, типов данных и сущностей

### Таблица user

Поля и размеры:

- id - 4 байт
- username - 21 байт
- registered\_at - 8 байт
- status - 11 байт
- name - 51 байт
- surname - 51 байт

- password - 101 байт

### **Таблица color**

Поля и размеры:

- id - 4 байт
- name - 51 байт

### **Таблица post**

Поля и размеры:

- id - 4 байт
- author\_id - 4 байт
- path\_to\_preview - 51 байт
- scheme - 990000 байт
- legend - 990000 байт
- type - 11 байт
- scheme\_name - 51 байт
- description - 513 байт
- comment - 513 байт
- colors\_count - 4 байт
- threads\_length - 4 байт
- created\_at - 8 байт
- updated\_at 8 байт

### **Таблица comment**

Поля и размеры:

- id - 4 байт
- post\_id - 4 байт
- author\_id - 4 байт
- text - 513 байт

### **Таблица action**

Поля и размеры:

- id - 4 байт
- description - 65 байт
- length - 4 байт

### **Связующие таблицы**

#### **Таблица post\_reaction**

Поля и размеры:

- post\_id - 4 байт
- user\_id - 4 байт
- is\_like - 1 байт

#### **Таблица post\_color**

Поля и размеры:

- post\_id - 4 байт
- color\_id - 4 байт

### **Оценка объема информации**

#### **Средний размер данных, хранимой в модели:**

user:  $4 + 21 + 8 + 11 + 51 + 51 + 101 = 247$  байт

color:  $4 + 51 = 55$  байт

post:  $4 + 4 + 51 + 990000 + 990000 + 11 + 51 + 513 + 513 + 4 + 4 + 8 + 8 = 1981171$  байт

comment:  $4 + 4 + 4 + 513 = 525$  байт

action:  $4 + 65 + 4 = 73$  байт

#### **Связующие таблицы:**

post\_reaction:  $4 + 4 + 1 = 9$  байт



post\_color:  $4 + 4 = 8$  байт

### Расчет для N постов

| Компонент             | Формула               | На 1 пост | на 10000 постов |
|-----------------------|-----------------------|-----------|-----------------|
| Основные данные поста | $1981171 * N$         | 1,88 МБ   | 18,4 ГБ         |
| Автор                 | $247 * N$             | 247 Б     | 2,23 МБ         |
| Итого                 | $(1981171 + 247) * N$ | 1,88 МБ   | 18,4 ГБ         |

### Избыточность данных

Избыточные данные

1. Избыточные данные

Комментарий 513 байт

Дата создания 8 байт

Дата обновления 8 байт

2. Чистый объем

$(1.981.171 - 513 - 8 - 8) * N = 1.980.642 * N$

3. Коэффициент избыточности

$1.982.071 / 1.980.642 = 1,0007$

### Направление роста модели

При добавлении новой схемы увеличиваются следующие таблицы: Post (+ 1981171 байт) color (+ 55 байт) Если это первый пост пользователя, то user (+ 247 байт) comment (+ 525 байт)

## Примеры запросов

### Получить всех активных пользователей

```
SELECT id, username, name, surname  
FROM user  
WHERE status = 'active';
```

### Найти все схемы вышивки с количеством цветов больше 3

```
SELECT id, scheme_name, colors_count  
FROM post  
WHERE colors_count > 3  
ORDER BY created_at DESC;
```

### Получить все комментарии к посту с ID=1

```
SELECT c.text, u.username  
FROM comment c  
JOIN user u ON c.author_id = u.id  
WHERE c.post_id = 1;
```

### Получить все посты с именами авторов

```
SELECT p.id, p.scheme_name, u.username AS author  
FROM post p  
JOIN user u ON p.author_id = u.id;
```

### Найти все цвета, используемые в посте с ID=1

```
SELECT c.name  
FROM color c  
JOIN post_color pc ON c.id = pc.color_id  
WHERE pc.post_id = 1;
```

### Получить посты с количеством лайков

```
SELECT p.id, p.scheme_name,
```

```
COUNT(CASE WHEN pr.is_like = 1 THEN 1 END) AS likes_count
FROM post p
LEFT JOIN post_reaction pr ON p.id = pr.post_id
GROUP BY p.id;
```

#### **Добавить нового пользователя**

```
INSERT INTO user (id, username, registered_at, status, name, surname,
password)
VALUES (5, 'new_user', UNIX_TIMESTAMP(), 'active', 'New', 'User',
'hashed_pass_xyz');
```

#### **Добавить реакцию к посту**

```
INSERT INTO post_reaction (post_id, user_id, is_like)
VALUES (2, 3, 1);
```

#### **Обновить статус пользователя**

```
UPDATE user
SET status = 'active'
WHERE id = 4;
```

#### **Обновить описание поста**

```
UPDATE post
SET description = 'Updated floral pattern description'
WHERE id = 1;
```

#### **Удалить комментарий**

```
DELETE FROM comment
WHERE id = 4;
```

#### **Удалить все реакции пользователя 3**

```
DELETE FROM post_reaction
WHERE user_id = 3;
```

## **СРАВНЕНИЕ МОДЕЛЕЙ**

### **Удельный объем информации**

При сравнении реляционной и нереляционной моделей хранения данных для системы схем вышивки можно выделить несколько ключевых аспектов. NoSQL-подход демонстрирует лучшую эффективность использования памяти - один документ с полной информацией о схеме вышивки вместе с изображением занимает примерно 2,1 МБ. Такая экономия пространства достигается благодаря встраиванию всех связанных данных непосредственно в документ, включая массивы лайков/дизлайков и легенды цветов, а также отсутствию необходимости хранить отдельные таблицы связей. В реляционной модели аналогичный набор данных требует около 1,88 МБ для основной информации плюс дополнительные 2 МБ для хранения изображения отдельно, что в сумме дает приблизительно 4 МБ на одну схему. Это увеличение объема происходит из-за необходимости нормализации данных, хранения связей в отдельных таблицах и раздельного хранения медиафайлов.

### **Запросы по отдельным юзкейсам**

С точки зрения производительности запросов NoSQL-решение также имеет преимущества. Для получения полной информации о схеме достаточно одного запроса к коллекции posts, тогда как в реляционной модели требуется сложный запрос с 3-4 JOIN операциями между таблицами поста, автора, реакций и цветов. При загрузке комментариев к схеме NoSQL требует всего один запрос к коллекции comments с фильтрацией по post\_id, в то время как SQL-вариант предполагает либо JOIN между таблицами comment и user, либо два отдельных запроса. Добавление реакции пользователя в NoSQL выполняется одной атомарной операцией обновления документа, тогда как в реляционной модели для этого требуется транзакция из трех операций.

### **Вывод**

NoSQL-модель особенно эффективна для систем с преобладанием

операций чтения, так как позволяет получать все необходимые данные за минимальное количество запросов. Все содержимое схемы хранится в одном документе, что обеспечивает быстрый доступ и простоту масштабирования. Реляционная модель, несмотря на строгую структуру и лучшую поддержку целостности данных, проигрывает в производительности из-за необходимости выполнения сложных запросов с объединениями и множественных обращений к разным таблицам. Для приложений, где важна скорость чтения и гибкость структуры данных, NoSQL оказывается более практичным выбором, особенно при большом количестве запросов и относительно редких обновлениях.

## РАЗРАБОТАННОЕ ПРИЛОЖЕНИЕ

### Краткое описаниеИспользованные технологииСнимки экрана приложения

Данное веб-приложение предназначено для поиска просмотра и публикации схем для вышивания и вязания.

Сервис включает следующие ключевые возможности:

- Просмотр схем с конкретными инструкциями по созданию изделия.
- Просмотр различных инструментов и их сложности и требуемым материалам.
- Экспорт/импорт базы данных.
- Поиск пользователей с различными фильтрами и постов которые они сделали.

### Использованные технологии

Для реализации приложения были использованы следующие технологии:

- Frontend:
  - *Jinja* — библиотека для создания веб интерфейса на python с использованием html шаблонов;
- Backend:
  - *Python* - основной язык программирования;
  - *FastAPI* - высокопроизводительный веб-фреймворк для создания API;
  - *MongoDB* - NoSQL база данных;
  - *Docker* и *docker-compose* - контейнеризация приложения.

### Снимки экрана приложения

Привет, Анна Иванова!ВыйтиПоиск пользователейДействияСоздать постДобавить действиеИмпорт/Экспорт

Все схемы  
для рукоделия

Тип работы:Все типы

Цвета:

☐ red

☐ blue

☐ green

☐ black

☐ white

Длина ниток:

От

До

Кол-во цветов:

От

До

Дата создания:

ДД-ММ-ГГГГ

...

—

ДД-ММ-ГГГГ

...

Фильтр по лайкам:Мин - Макс

Автор поста:NoneNone

Сортировка:ДатаПо убыванию

Применить фильтрыСбросить

Показано 1 - 5 из 11

Страница:1 / 3На странице:5

подложка

Автор: Иван Петров | Дата: 15.05.2024 14:30 | Тип: вязание

1 лайков5 цветов1704 ед. ниток

чай поставьте



Цветы

Автор: Иван Петров | Дата: 15.05.2023 14:30 | Тип: вышивка

1 лайков5 цветов192 ед. ниток

Схема для вышивки цветов



Рис. 4 – Главная страница.

31

## подложка

Автор: [Иван Петров](#) | Дата: 2024-08-15 14:30 | Тип: вязание



Рис. 5 – Часть страницы просмотра поста.



Описание

чай поставьте

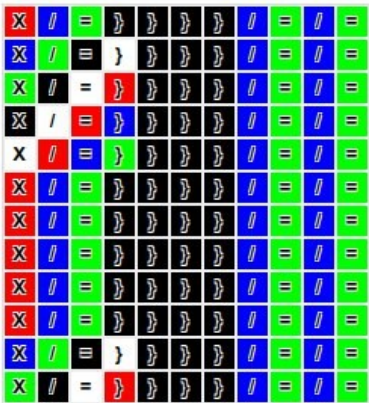
Комментарий автора

Рекомендую канву Aida 16

Легенда стежков

X - крестик / - полукрест = - гобеленовый | - стебельчатый } - левый стежок \* - полуторный  
^ - тамбурный ; - расколотый @ - французский \$ - гладью % - назад вперед

Схема вышивки



Комментарии (1)

Напишите ваш комментарий...

Отправить комментарий

Анна Иванова

2025-06-25 07:27

fahik

Рис. 6 – Часть страницы просмотра поста.

## Просмотр стежков

Поиск по описанию:

Фильтр по длине:

[Применить фильтры](#) [Сбросить все](#) [Сортировка: По убыванию](#)

|                               |
|-------------------------------|
| Длина: 18 ед.<br>левый стежок |
| Длина: 17 ед.<br>полуторный   |
| Длина: 16 ед.<br>тамбурный    |
| Длина: 15 ед.<br>гобеленовый  |
| Длина: 14 ед.<br>расколотый   |
| Длина: 13 ед.<br>французский  |
| Длина: 12 ед.<br>гладью       |
| Длина: 11 ед.<br>назад вперед |

Рис. 7 – Страница просмотра стежков.

## Управление базой данных

The screenshot shows a web interface for database management. It is divided into three main sections: Export, Import, and Delete. The 'Export' section has a button labeled 'Экспорт в JSON'. The 'Import' section has a file selection area with the text 'Выберите файл' and 'Файл не выбран', and a button labeled 'Импорт из JSON'. The 'Delete' section is highlighted with a red background and contains a button labeled 'Удалить все данные'.

Рис. 8 – Страница импорта экспорта.

## ВЫВОДЫ

### Достигнутые результаты

Была реализована страница поиска всех схем с пагинацией(серверной), фильтрами, сортировкой и поиском (по дате, типу длине нити, количеству цветов и конкретным цветам), страницу с сортировкой всех действий(стежков и крючков, называется действия), отдельная страница в которой информация о каждой схеме, превью, легенда, описание и комментарий автора, а так же рендер самой схемы с обозначением какие действия делать и какими цветами. за основу взял базу данных ранее мной реализованную в рамках общего проекта. Есть отдельная страница экспорта импорта базы в json файл, для удобства тестирования сделал кнопку очистки базы данных.

### Недостатки и пути для улучшения полученного решения

Можно сильно улучшить интерфейс пользователя и добавить много функций например редактирование постов и данных. Приложение не осуществляет весь первоначальный функционал из-за разобщенности команды. Путем решения будет тщательный анализ и доработка всех недостатков.

## **Будущее развитие решения**

Можно сильно улучшить интерфейс пользователя и добавить много функций например редактирование постов и данных. Доработать новые страницы, добавить какой нибудь форум для обсуждения, чат с экспертами. Добавить статистику для администраторов.

## ПРИЛОЖЕНИЯ

### Документация по сборке и развертыванию приложения

1. Склонировать репозиторий.
2. Перейти в корневую директорию проекта.
3. Собрать docker-контейнер командой ``docker compose build --no-cache``.
4. Запустить docker-контейнер с приложением командой ``docker compose up``.
5. Открыть веб-приложение в браузере по адресу: `http://localhost:8081`.

### Инструкция для пользователя

#### Просмотр страниц и поиск.

1. На главной странице вы можете искать схемы применяя различные фильтры.
2. Есть страница «действия» для поиска действий.
3. есть страница «Поиск пользователей» для поиска пользователей с различными фильтрами.
4. Поиск доступен и без авторизации, но можно авторизоваться зайдя на страницу авторизации в верхней навигационной панели.

#### Выход из системы

1. Для выхода нажмите на значок выхода в верхней навигационной панели.

## ЛИТЕРАТУРА

1. Ссылка на GitHub репозиторий. – [Электронный ресурс]. – URL: [https://github.com/moevm/nosql1h25-needlework/tree/Sysuev\\_Mikhail](https://github.com/moevm/nosql1h25-needlework/tree/Sysuev_Mikhail)
2. Ссылка на макеты в Figma. – [Электронный ресурс]. – URL: <https://www.figma.com/design/5b2pXrdkpsEf1mgQG9nRxU/nosql-frontend?node-id=0-1&p=f&t=mnle89OZyYr7prFz-0>.
3. Документация Jinja. – [Электронный ресурс]. – URL: <https://jinja.palletsprojects.com/en/stable/> (дата обращения 01.06.2025).
4. Документация MongoDB. – [Электронный ресурс]. – URL: <https://mongodb.com/> (дата обращения 01.06.2025).